

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Akmallullail Sya'ban NIM. 2310817310010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Akmallullail Sya'ban
NIM : 2310817310010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom
NIP. 198205082008011010

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	6
B. Output Program	34
C. Pembahasan	39
Jetpack Compose	39
D. Tautan Git	51

DAFTAR GAMBAR

Gambar 1. Tampilan Aplikasi Ketika Offline	34
Gambar 2. Tampilan Aplikasi Ketika Offline dalam Light Mode	35
Gambar 3. Tampilan Tombol Ubah Dark/Light Mode.....	35
Gambar 4. Tampilan Tombol Ubah Dark/Light Mode.....	36
Gambar 5. Tampilan Aplikasi Saat Retry Ditekan dan Online dalam Light Mode.....	36
Gambar 6. Tombol Detail Ditekan dalam Light Mode	37
Gambar 7. Tampilan Aplikasi Saat Retry Ditekan dan Online dalam Dark Mode	37
Gambar 8. Tombol Detail Ditekan dalam Dark Mode	38
Gambar 9. Tombol TMDB Saat Ditekan.....	38

DAFTAR TABEL

Tabel 1. Source Code AppDatabase.kt Compose	6
Tabel 2. Source Code MovieDao.kt Compose	7
Tabel 3. Source Code MovieEntity.kt Compose	7
Tabel 4. Source Code SettingManager.kt Compose	8
Tabel 5. Source Code MovieApi.kt Compose	9
Tabel 6. Source Code MovieResponse.kt Compose	9
Tabel 7. Source Code ListMovie.kt Compose	10
Tabel 8. Source Code RetrofitClient.kt Compose	10
Tabel 9. Source Code TmdbApiService.kt Compose	11
Tabel 10. Source Code MovieRepository.kt Compose	12
Tabel 11. Source Code DetailActivity.kt Compose	14
Tabel 12. Source Code MovieListScreen.kt Compose	17
Tabel 13. Source Code SettingScreen.kt Compose	25
Tabel 14. Source Code Constants.kt Compose	27
Tabel 15. Source Code Resource.kt Compose	27
Tabel 16. Source Code RetrofitClient.kt Compose	28
Tabel 17. Source Code MovieViewModelFactory.kt Compose	31
Tabel 18. Source Code MovieViewModelFactory.kt Compose	31
Tabel 19. Source Code MainActivity.kt Compose	32

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

Compose:

data/local/AppDatabase.kt

Tabel 1. Source Code AppDatabase.kt Compose

1	package com.example.mlistcompose.data.local
2	
3	import android.content.Context
4	import androidx.room.Database
5	import androidx.room.Room
6	import androidx.room.RoomDatabase
7	
8	@Database(entities = [MovieEntity::class], version = 1, exportSchema = false)
9	abstract class AppDatabase : RoomDatabase() {
10	abstract fun movieDao(): MovieDao
11	
12	companion object {
13	@Volatile
14	private var INSTANCE: AppDatabase? = null

```

15
16         fun getDatabase(context: Context): AppDatabase {
17             return INSTANCE ?: synchronized(this) {
18                 val instance = Room.databaseBuilder(
19                     context.applicationContext,
20                     AppDatabase::class.java,
21                     "movie_database"
22                 ).build()
23                 INSTANCE = instance
24                 instance
25             }
26         }
27     }
28 }

```

data/local/MovieDao.kt

Tabel 2. Source Code MovieDao.kt Compose

```

1 package com.example.mlistcompose.data.local
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.OnConflictStrategy
6 import androidx.room.Query
7 import kotlinx.coroutines.flow.Flow
8
9 @Dao
10 interface MovieDao {
11     @Insert(onConflict = OnConflictStrategy.REPLACE)
12     suspend fun insertMovies(movies: List<MovieEntity>)
13
14     @Query("SELECT * FROM movies ORDER BY releaseDate
15     DESC")
16     fun getAllMovies(): Flow<List<MovieEntity>>
17
18     @Query("DELETE FROM movies")
19     suspend fun deleteAllMovies()
20 }

```

data/local/MovieEntity.kt

Tabel 3. Source Code MovieEntity.kt Compose

```

1 package com.example.mlistcompose.data.local
2
3 import androidx.room.Entity

```

```

4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "movies")
7 data class MovieEntity(
8     @PrimaryKey val id: Int,
9     val title: String,
10    val posterPath: String?,
11    val releaseDate: String?,
12    val overview: String?,
13    val voteAverage: Double?
14 )

```

data/local/preferences/SettingManager.kt

Tabel 4. Source Code SettingManager.kt Compose

```

1 package com.example.mlistcompose.data.preferences
2
3 import android.content.Context
4 import android.content.SharedPreferences
5 import androidx.core.content.edit
6 import kotlinx.coroutines.channels.awaitClose
7 import kotlinx.coroutines.flow.Flow
8 import kotlinx.coroutines.flow.callbackFlow
9
10 class SettingsManager(context: Context) {
11     private val prefs: SharedPreferences =
12         context.getSharedPreferences("app_settings",
13             Context.MODE_PRIVATE)
14
15     companion object {
16         const val KEY_DARK_MODE = "dark_mode"
17     }
18
19     fun getDarkModeEnabledAsFlow(): Flow<Boolean> =
20         callbackFlow {
21             val listener =
22                 SharedPreferences.OnSharedPreferenceChangeListener { _,
23                     key ->
24                         if (key == KEY_DARK_MODE) {
25                             trySend(isDarkModeEnabled())
26                         }
27                     }
28
29             prefs.registerOnSharedPreferenceChangeListener(listener)
30             trySend(isDarkModeEnabled())
31             awaitClose {

```



```

    prefs.unregisterOnSharedPreferenceChangeListener(listener)
  }
26   }
27
28   fun isDarkModeEnabled(): Boolean {
29       return prefs.getBoolean(KEY_DARK_MODE, false)
30   }
31
32   fun setDarkModeEnabled(isEnabled: Boolean) {
33       prefs.edit { putBoolean(KEY_DARK_MODE, isEnabled)
34   }
35   }
  }
}

```

model/network/MovieApi.kt

Tabel 5. Source Code MovieApi.kt Compose

```

1  package com.example.mlistcompose.model.network
2
3  import kotlinx.serialization.SerialName
4  import kotlinx.serialization.Serializable
5
6  @Serializable
7  data class MovieApi(
8      val id: Int,
9      val title: String?,
10     @SerialName("poster_path") val posterPath: String?,
11     @SerialName("release_date") val releaseDate:
12     String?,
13     val overview: String?,
14     @SerialName("vote_average") val voteAverage: Double?
15 )

```

model/network/MovieResponse.kt

Tabel 6. Source Code MovieResponse.kt Compose

```

1  package com.example.mlistcompose.model.network
2
3  import kotlinx.serialization.SerialName
4  import kotlinx.serialization.Serializable
5
6  @Serializable
7  data class MovieResponse(
8      val page: Int,
9      val results: List<MovieApi>,

```

10	@SerializedName("total_pages") val totalPages: Int,
11	@SerializedName("total_results") val totalResults: Int
12	)

model/ListMovie.kt

Tabel 7. Source Code ListMovie.kt Compose

1	package com.example.mlistcompose.model
2	
3	data class ListMovie(4 val id: Int, 5 val title: String, 6 val imageUrl: String?, 7 val releaseDate: String?, 8 val description: String?, 9 val voteAverage: Double? 10 >)

network/RetrofitClient.kt

Tabel 8. Source Code RetrofitClient.kt Compose

1	package com.example.mlistcompose.network
2	
3	import com.example.mlistcompose.utils.Constants
4	import com.jakewharton.retrofit2.converter.kotlinx.serialization. on.asConverterFactory
5	import kotlinx.serialization.ExperimentalSerializationApi
6	import kotlinx.serialization.json.Json
7	import okhttp3.Interceptor
8	import okhttp3.MediaType.Companion.toMediaType
9	import okhttp3.OkHttpClient
10	import okhttp3.logging.HttpLoggingInterceptor
11	import retrofit2.Retrofit
12	
13	object RetrofitClient {
14	
15	private val json = Json { ignoreUnknownKeys = true }
16	
17	private val loggingInterceptor = HttpLoggingInterceptor().apply { 18 setLevel(HttpLoggingInterceptor.Level.BODY) 19 }

```

20
21     private val authInterceptor = Interceptor { chain -
22 >         val originalRequest = chain.request()
23         val newUrl = originalRequest.url.newBuilder()
24             .addQueryParameter("api_key",
25 Constants.TMDB_API_KEY)
26             .build()
27         val newRequest =
28 originalRequest.newBuilder().url(newUrl).build()
29         chain.proceed(newRequest)
30     }
31
32     private val okHttpClient = OkHttpClient.Builder()
33         .addInterceptor(authInterceptor)
34         .addInterceptor(loggingInterceptor)
35         .build()
36
37     @OptIn(ExperimentalSerializationApi::class)
38     val tmdbApiService: TmdbApiService by lazy {
39         Retrofit.Builder()
40             .baseUrl(Constants.TMDB_BASE_URL)
41             .client(okHttpClient)
42
43 .addConverterFactory(json.asConverterFactory("applicati
44 on/json".toMediaType()))
45         .build()
46         .create(TmdbApiService::class.java)
47     }
48 }

```

network/TmdbApiService.kt

Tabel 9. Source Code TmdbApiService.kt Compose

```

1 package com.example.mlistcompose.network
2
3 import
4 com.example.mlistcompose.model.network.MovieResponse
5 import retrofit2.http.GET
6 import retrofit2.http.Query
7
8 interface TmdbApiService {
9     @GET("movie/popular")
10     suspend fun getPopularMovies(
11         @Query("language") language: String = "en-US",

```

11	@Query("page") page: Int = 1
12	): MovieResponse
13	}

repository/MovieRepository.kt

Tabel 10. Source Code MovieRepository.kt Compose

1	package com.example.mlistcompose.repository
2	
3	import com.example.mlistcompose.data.local.MovieDao
4	import com.example.mlistcompose.data.local.MovieEntity
5	import com.example.mlistcompose.model.network.MovieApi
6	import com.example.mlistcompose.network.TmdbApiService
7	import com.example.mlistcompose.utils.Resource
8	import kotlinx.coroutines.flow.Flow
9	import kotlinx.coroutines.flow.flow
10	import kotlinx.coroutines.flow.firstOrNull
11	import timber.log.Timber
12	
13	class MovieRepository(
14	private val apiService: TmdbApiService,
15	private val movieDao: MovieDao
16	) {
17	fun getPopularMovies(
18	fetchFromRemote: Boolean
19	): Flow<Resource<List<MovieEntity>>> = flow {
20	emit(Resource.Loading())
21	
22	val localMovies =
23	movieDao.getAllMovies().firstOrNull()
24	if (!localMovies.isNullOrEmpty()) {
25	emit(Resource.Success(localMovies))
26	Timber.d("Loaded \${localMovies.size} movies
27	from cache.")
28	}
29	
30	val shouldLoadFromApi = fetchFromRemote
31	localMovies.isNullOrEmpty()
32	
33	if (shouldLoadFromApi) {
34	try {
35	Timber.d("Fetching movies from API...")
	val response =
	apiService.getPopularMovies()
	val remoteMovies = response.results
	val movieEntities = remoteMovies.map {

```

36 it.toMovieEntity() }
37         movieDao.deleteAllMovies()
38         movieDao.insertMovies(movieEntities)
39         Timber.d("Fetched and cached
    ${movieEntities.size} movies from API.")
40
41         val newLocalMovies =
42 movieDao.getAllMovies().firstOrNull()
43         if (newLocalMovies != null) {
44
45 emit(Resource.Success(newLocalMovies))
46         } else {
47             emit(Resource.Error("Failed to
48 retrieve new data from cache after API call.))
49         }
50
51         } catch (e: Exception) {
52             Timber.e(e, "Error fetching movies from
53 API: ${e.message}")
54             if (localMovies.isNullOrEmpty()) {
55                 emit(Resource.Error("Couldn't reach
56 server. Check your internet connection.", emptyList()))
57             } else {
58                 emit(Resource.Error("Failed to
59 update data. Displaying cached data.", localMovies))
60             }
61         }
62     } else {
63         if (localMovies.isNullOrEmpty()) {
64             emit(Resource.Error("No cached data
65 available and not fetching from remote.))
66         }
67     }
68 }
69
70 private fun MovieApi.toMovieEntity(): MovieEntity {
71     return MovieEntity(
72         id = this.id,
73         title = this.title ?: "No Title",
74         posterPath = this.posterPath,
75         releaseDate = this.releaseDate,
76         overview = this.overview,
77         voteAverage = this.voteAverage
78     )
79 }
80

```

screen/DetailActivity*Tabel 11. Source Code DetailActivity.kt Compose*

1	package com.example.mlistcompose.screen
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.layout.*
7	import androidx.compose.foundation.rememberScrollState
8	import androidx.compose.foundation.verticalScroll
9	import androidx.compose.material3.MaterialTheme
10	import androidx.compose.material3.Surface
11	import androidx.compose.material3.Text
12	import androidx.compose.ui.Alignment
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.layout.ContentScale
15	import androidx.compose.ui.text.TextStyle
16	import androidx.compose.ui.text.font.FontWeight
17	import androidx.compose.ui.unit.dp
18	import androidx.compose.ui.unit.sp
19	import coil.compose.AsyncImage
20	import
	com.example.mlistcompose.ui.theme.MListComposeTheme
21	import
	com.example.mlistcompose.data.preferences.SettingsManager
22	
23	class DetailActivity : ComponentActivity() {
24	override fun onCreate(savedInstanceState: Bundle?)
	{
25	super.onCreate(savedInstanceState)
26	
27	val title = intent.getStringExtra("title") ?:
	"N/A"
28	val imageUrl =
	intent.getStringExtra("imageUrl") ?: ""
29	val releaseDate =
	intent.getStringExtra("releaseDate") ?: "N/A"
30	val overview =
	intent.getStringExtra("description") ?: "No
	description available."
31	val voteAverage =
	intent.getDoubleExtra("voteAverage", 0.0)
32	


```

69         text = "Release Date:
$releaseDate",
70         style = TextStyle(
71             fontSize = 18.sp,
72             color =
MaterialTheme.colorScheme.onBackground
73         ),
74         modifier =
Modifier.padding(bottom = 4.dp)
75     )
76     Text(
77         text = "Rating:
%.1f/10".format(voteAverage),
78         style = TextStyle(
79             fontSize = 18.sp,
80             color =
MaterialTheme.colorScheme.onBackground
81         ),
82         modifier =
Modifier.padding(bottom = 16.dp)
83     )
84     Text(
85         text = "Plot :",
86         style = TextStyle(
87             fontSize = 20.sp,
88             fontWeight =
FontWeight.Bold,
89             color =
MaterialTheme.colorScheme.onBackground
90         )
91     )
92     Spacer(modifier =
Modifier.height(8.dp))
93     Text(
94         text = overview,
95         style = TextStyle(
96             fontSize = 16.sp,
97             color =
MaterialTheme.colorScheme.onBackground
98         )
99     )
100 }
101 }
102 }
103 }
104 }
105 }

```


--	--

screen/MovieListScreen.kt

Tabel 12. Source Code MovieListScreen.kt Compose

1	package com.example.mlistcompose.screen
2	
3	import android.content.Intent
4	import androidx.compose.foundation.layout.Arrangement
5	import androidx.compose.foundation.layout.Column
6	import androidx.compose.foundation.layout.PaddingValues
7	import androidx.compose.foundation.layout.Row
8	import androidx.compose.foundation.layout.Spacer
9	import androidx.compose.foundation.layout.fillMaxSize
10	import androidx.compose.foundation.layout.fillMaxWidth
11	import androidx.compose.foundation.layout.height
12	import androidx.compose.foundation.layout.padding
13	import androidx.compose.foundation.layout.size
14	import androidx.compose.foundation.layout.width
15	import
	androidx.compose.foundation.layout.wrapContentHeight
16	import
	androidx.compose.foundation.layout.wrapContentWidth
17	import androidx.compose.foundation.lazy.LazyColumn
18	import androidx.compose.foundation.lazy.items
19	import
	androidx.compose.foundation.shape.RoundedCornerShape
20	import androidx.compose.material3.Button
21	import androidx.compose.material3.Card
22	import androidx.compose.material3.CardDefaults
23	import
	androidx.compose.material3.CircularProgressIndicator
24	import androidx.compose.material3.MaterialTheme
25	import androidx.compose.material3.Text
26	import androidx.compose.runtime.Composable
27	import androidx.compose.runtime.collectAsState
28	import androidx.compose.runtime.getValue
29	import androidx.compose.ui.Alignment
30	import androidx.compose.ui.Modifier
31	import androidx.compose.ui.draw.clip
32	import androidx.compose.ui.graphics.Color
33	import androidx.compose.ui.layout.ContentScale
34	import androidx.compose.ui.platform.LocalContext
35	import androidx.compose.ui.text.TextStyle
36	import androidx.compose.ui.text.font.FontWeight

```

37 import androidx.compose.ui.unit.dp
38 import androidx.compose.ui.unit.sp
39 import androidx.core.net.toUri
40 import androidx.navigation.NavController
41 import coil.compose.AsyncImage
42 import
43 com.example.mlistcompose.viewmodel.MovieViewModel
44 import com.example.mlistcompose.model.ListMovie
45 import com.example.mlistcompose.utils.Resource
46 import androidx.compose.material3.Scaffold
47 import androidx.compose.material3.TopAppBar
48 import androidx.compose.material3.TopAppBarDefaults
49 import androidx.compose.material3.IconButton
50 import androidx.compose.material3.Icon
51 import androidx.compose.material.icons.Icons
52 import androidx.compose.material.icons.filled.Settings
53 import
54 androidx.compose.material3.ExperimentalMaterial3Api
55 import com.example.mlistcompose.utils.Constants
56
57 @OptIn(ExperimentalMaterial3Api::class,
58 ExperimentalMaterial3Api::class)
59 @Composable
60 fun MovieListScreen(
61     viewModel: MovieViewModel,
62     navController: NavController
63 ) {
64     val moviesResource by
65     viewModel.movies.collectAsState()
66     val navigateToUrl by
67     viewModel.navigateToUrl.collectAsState()
68     val navigateToDetail by
69     viewModel.navigateToDetail.collectAsState()
70     val context = LocalContext.current
71
72     navigateToUrl?.let { url ->
73         val intent = Intent(Intent.ACTION_VIEW,
74             url.toUri())
75         context.startActivity(intent)
76         viewModel.onUrlNavigated()
77     }
78
79     navigateToDetail?.let { movie ->
80         val intent = Intent(context,
81             DetailActivity::class.java).apply {
82             putExtra("id", movie.id)
83             putExtra("title", movie.title)
84         }
85     }

```

```

77         putExtra("imageUrl", movie.imageUrl)
78         putExtra("releaseDate", movie.releaseDate)
79         putExtra("description", movie.description)
80         putExtra("voteAverage", movie.voteAverage)
81     }
82     context.startActivity(intent)
83     viewModel.onDetailNavigated()
84 }
85
86 Scaffold(
87     topBar = {
88         TopAppBar(
89             title = { Text("Film Populer", color =
MaterialTheme.colorScheme.onPrimary) },
90             actions = {
91                 IconButton(onClick = {
navController.navigate("settings") }) {
92                     Icon(
93                         imageVector =
Icons.Filled.Settings,
94                         contentDescription =
"Settings",
95                         tint =
MaterialTheme.colorScheme.onPrimary
96                     )
97                 }
98             },
99             colors =
TopAppBarDefaults.topAppBarColors(
100                 containerColor =
MaterialTheme.colorScheme.primary,
101                 titleContentColor =
MaterialTheme.colorScheme.onPrimary
102             )
103         )
104     },
105     containerColor =
MaterialTheme.colorScheme.background
106 ) { paddingValues ->
107     Column(
108         modifier = Modifier
109             .padding(paddingValues)
110             .fillMaxSize()
111     ) {
112         when (moviesResource) {
113             is Resource.Loading -> {
114                 Column(

```

```

115         modifier =
116         Modifier.fillMaxSize(),
117         verticalArrangement =
118         Arrangement.Center,
119         horizontalAlignment =
120         Alignment.CenterHorizontally
121     ) {
122         CircularProgressIndicator(color
123         = Color.White)
124         Spacer(modifier =
125         Modifier.height(8.dp))
126         Text("Loading movies...", color
127         = Color.White)
128     }
129 }
130 is Resource.Success -> {
131     val movieList = moviesResource.data
132     ?: emptyList()
133     LazyColumn(
134         contentPadding =
135         PaddingValues(16.dp),
136         verticalArrangement =
137         Arrangement.spacedBy(16.dp)
138     ) {
139         items(movieList) { movie ->
140             MovieItems(
141                 movie = movie,
142                 onDetailClicked = {
143                     viewModel.onDetailClicked(movie) },
144                 onTmdbClicked = { url -
145                     > viewModel.onImdbClicked(url) }
146             )
147         }
148     }
149 }
150 is Resource.Error -> {
151     val errorMessage =
152     moviesResource.message ?: "An unknown error occurred"
153     val cachedData =
154     moviesResource.data ?: emptyList()
155     if (cachedData.isNotEmpty()) {
156         LazyColumn(
157             contentPadding =
158             PaddingValues(16.dp),
159             verticalArrangement =
160             Arrangement.spacedBy(16.dp),

```

```

147         modifier =
Modifier.fillMaxSize()
148     ) {
149         item {
150             Column(
151                 modifier = Modifier
152                     .fillMaxWidth()
153                     .padding(bottom
= 16.dp),
154                 horizontalAlignment
= Alignment.CenterHorizontally
155             ) {
156                 Text(
157                     "Error:
$errorMessage",
158                     color =
Color.Red,
159                     fontSize =
18.sp
160                 )
161                 Text(
162                     "Displaying
cached data.",
163                     color =
Color.Gray,
164                     fontSize =
14.sp
165                 )
166                 Spacer(modifier =
Modifier.height(16.dp))
167                 Button(onClick = {
viewModel.fetchMovies(true) }) {
168                     Text("Retry",
color = Color.White)
169                 }
170             }
171         }
172         items(cachedData) { movie -
>
173             MovieItems(
174                 movie = movie,
175                 onDetailClicked = {
viewModel.onDetailClicked(movie) },
176                 onTmdbClicked = {
url -> viewModel.onImdbClicked(url) }
177             )
178         }

```

```

179         }
180     } else {
181         Column(
182             modifier =
183             Modifier.fillMaxSize(),
184             verticalArrangement =
185             Arrangement.Center,
186             horizontalAlignment =
187             Alignment.CenterHorizontally
188         ) {
189             Text("Error:
190             $errorMessage", color = Color.Red, fontSize = 18.sp)
191             Spacer(modifier =
192             Modifier.height(16.dp))
193             Button(onClick = {
194             viewModel.fetchMovies(true) }) {
195                 Text("Retry", color =
196                 Color.White)
197             }
198         }
199     }
200 }
201
202 @Composable
203 fun MovieItems(
204     movie: ListMovie,
205     onDetailClicked: () -> Unit,
206     onTmdbClicked: (String) -> Unit
207 ) {
208     Card(
209         shape = RoundedCornerShape(16.dp),
210         modifier = Modifier
211             .fillMaxWidth()
212             .wrapContentSize(),
213         colors = CardDefaults.cardColors(containerColor
214         = MaterialTheme.colorScheme.surface),
215         elevation = CardDefaults.cardElevation(4.dp)
216     ) {
217         Row(
218             modifier = Modifier
219                 .padding(16.dp)
220                 .fillMaxWidth()
221         ) {

```

```

220         AsyncImage(
221             model = movie.imageUrl,
222             contentDescription = movie.title,
223             contentScale = ContentScale.Crop,
224             modifier = Modifier
225                 .size(width = 90.dp, height =
200.dp)
226                 .clip(RoundedCornerShape(8.dp))
227                 .align(Alignment.CenterVertically)
228         )
229         Spacer(modifier = Modifier.width(12.dp))
230         Column(
231             modifier = Modifier
232                 .fillMaxWidth()
233                 .align(Alignment.CenterVertically)
234         ) {
235             Row(
236                 modifier = Modifier
237                     .fillMaxWidth()
238                     .padding(8.dp)
239             ) {
240                 Text(
241                     text = movie.title,
242                     style = TextStyle(
243                         fontSize = 26.sp,
244                         fontWeight =
FontWeight.Bold,
245                         color =
MaterialTheme.colorScheme.onSurface
246                     ),
247                     modifier = Modifier
248                         .padding(bottom = 4.dp)
249                         .weight(1f)
250                 )
251                 Text(
252                     text =
movie.releaseDate?.substringBefore("-") ?: "N/A",
253                     style = TextStyle(
254                         fontSize = 20.sp,
255                         color =
MaterialTheme.colorScheme.onSurface
256                     ),
257                     modifier = Modifier
258                         .padding(bottom = 4.dp)
259                 )
                .wrapContentWidth(Alignment.End)

```

```

260 .align(Alignment.CenterVertically)
261     )
262     }
263     Row(
264         modifier = Modifier.fillMaxWidth()
265     ) {
266         Text(
267             text = "Plot :",
268             style = TextStyle(
269                 fontSize = 16.sp,
270                 fontWeight =
271                     FontWeight.Bold,
272                     color =
273                     MaterialTheme.colorScheme.onSurface
274                 )
275             )
276             Spacer(modifier =
277                 Modifier.width(4.dp) )
278             Text(
279                 text =
280                 (movie.description?.take(100) + "..."),
281                 style = TextStyle(
282                     fontSize = 16.sp,
283                     color =
284                     MaterialTheme.colorScheme.onSurface
285                 ),
286                 modifier =
287                 Modifier.padding(bottom = 8.dp)
288             )
289         }
290         Row(
291             modifier = Modifier.fillMaxWidth(),
292             horizontalArrangement =
293             Arrangement.End
294         ) {
295             Button(onClick = {
296                 val tmdbUrl =
297                 Constants.TMDB_MOVIE_WEB_BASE_URL + movie.id
298                 onTmdbClicked(tmdbUrl)
299             }) {
300                 Text("TMDB", color =
301                     MaterialTheme.colorScheme.onPrimary)
302             }
303             Spacer(modifier =
304                 Modifier.width(8.dp) )
305             Button(onClick = onDetailClicked) {
306                 Text("Detail", color =

```



```

297 MaterialTheme.colorScheme.onPrimary)
298     }
299     }
300     }
301     }
302 }
303 }

```

screen/SettingScreen.kt

Tabel 13. Source Code SettingScreen.kt Compose

```

1  package com.example.mlistcompose.screen
2
3  import androidx.compose.foundation.layout.*
4  import androidx.compose.material.icons.Icons
5  import
6  androidx.compose.material.icons.automirrored.filled.ArrowBack
7  import androidx.compose.material3.*
8  import androidx.compose.runtime.Composable
9  import androidx.compose.runtime.collectAsState
10 import androidx.compose.runtime.getValue
11 import androidx.compose.ui.Alignment
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.unit.dp
14 import androidx.navigation.NavController
15 import com.example.mlistcompose.viewmodel.MovieViewModel
16
17 @OptIn(ExperimentalMaterial3Api::class)
18 @Composable
19 fun SettingsScreen(
20     navController: NavController,
21     viewModel: MovieViewModel
22 ) {
23     val isDarkMode by
24     viewModel.isDarkModeEnabled.collectAsState()
25
26     Scaffold(
27         topBar = {
28             TopAppBar(
29                 title = { Text("Settings", color =
30                 MaterialTheme.colorScheme.onPrimary) },
31                 navigationIcon = {
32                     IconButton(onClick = {

```

```

navController.popBackStack() )) {
30         Icon(
31             imageVector =
Icons.AutoMirrored.Filled.ArrowBack,
32             contentDescription = "Back",
33             tint =
MaterialTheme.colorScheme.onPrimary
34         )
35     },
36     ),
37     colors = TopAppBarDefaults.topAppBarColors(
38         containerColor =
MaterialTheme.colorScheme.primary,
39         titleContentColor =
MaterialTheme.colorScheme.onPrimary
40     )
41 )
42 },
43     containerColor = MaterialTheme.colorScheme.background
44 ) { paddingValues ->
45     Column(
46         modifier = Modifier
47             .fillMaxSize()
48             .padding(paddingValues)
49             .padding(16.dp),
50         horizontalAlignment =
Alignment.CenterHorizontally,
51         verticalArrangement = Arrangement.Top
52     ) {
53         Row(
54             modifier = Modifier
55                 .fillMaxWidth()
56                 .padding(vertical = 8.dp),
57             verticalAlignment =
Alignment.CenterVertically,
58             horizontalArrangement =
Arrangement.SpaceBetween
59         ) {
60             Text(
61                 "Dark Mode",
62                 color =
MaterialTheme.colorScheme.onBackground,
63                 style =
MaterialTheme.typography.titleMedium
64             )
65             Switch(
66                 checked = isDarkMode,

```

```

67         onChangeChange = { isChecked ->
68             viewModel.setDarkMode(isChecked)
69         },
70         colors = SwitchDefaults.colors(
71             checkedThumbColor =
MaterialTheme.colorScheme.primary,
72             checkedTrackColor =
MaterialTheme.colorScheme.secondary,
73             uncheckedThumbColor =
MaterialTheme.colorScheme.onSurface,
74             uncheckedTrackColor =
MaterialTheme.colorScheme.surface
75         )
76     )
77 }
78 }
79 }
80 }

```

utils/Constants.kt

Tabel 14. Source Code Constants.kt Compose

```

1 package com.example.mlistcompose.utils
2
3 object Constants {
4     const val TMDB_API_KEY =
"01c9634cd909bf359669c502c2c8b378"
5     const val TMDB_BASE_URL =
"https://api.themoviedb.org/3/"
6     const val TMDB_IMAGE_BASE_URL =
"https://image.tmdb.org/t/p/w500/"
7     const val TMDB_MOVIE_WEB_BASE_URL =
"https://www.themoviedb.org/movie/"
8 }

```

utils/resource.kt

Tabel 15. Source Code Resource.kt Compose

```

1 package com.example.mlistcompose.utils
2
3 sealed class Resource<T>(val data: T? = null, val
4 message: String? = null) {
5     class Success<T>(data: T) : Resource<T>(data)
6     class Error<T>(message: String, data: T? = null) :
Resource<T>(data, message)

```

	class Loading<T>(data: T? = null) :
6	Resource<T>(data)
7	}

viewmodel/MovieViewModel.kt

Tabel 16. Source Code RetrofitClient.kt Compose

1	package com.example.mlistcompose.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import
	com.example.mlistcompose.data.preferences.SettingsManager
6	import com.example.mlistcompose.model.ListMovie
7	import
	com.example.mlistcompose.repository.MovieRepository
8	import com.example.mlistcompose.utils.Constants
9	import com.example.mlistcompose.utils.Resource
10	import kotlinx.coroutines.flow.MutableStateFlow
11	import kotlinx.coroutines.flow.SharingStarted
12	import kotlinx.coroutines.flow.StateFlow
13	import kotlinx.coroutines.flow.asStateFlow
14	import kotlinx.coroutines.flow.distinctUntilChanged
15	import kotlinx.coroutines.flow.stateIn
16	import kotlinx.coroutines.launch
17	import timber.log.Timber
18	
19	class MovieViewModel(
20	private val repository: MovieRepository,
21	private val settingsManager: SettingsManager
22	) : ViewModel() {
23	private val _movies =
	MutableStateFlow<Resource<List<ListMovie>>>(Resource.Loading()) // Resource of ListMovie
24	val movies: StateFlow<Resource<List<ListMovie>>> =
	_movies.asStateFlow()
25	
26	private val _navigateToUrl =
	MutableStateFlow<String?>(null)
27	val navigateToUrl: StateFlow<String?> =
	_navigateToUrl.asStateFlow()
28	
29	private val _navigateToDetail =
	MutableStateFlow<ListMovie?>(null)
30	val navigateToDetail: StateFlow<ListMovie?> =

```

31 _navigateToDetail.asStateFlow()
32     val isDarkModeEnabled: StateFlow<Boolean> =
33     settingsManager.getDarkModeEnabledAsFlow()
34         .distinctUntilChanged()
35         .stateIn(
36             scope = viewModelScope,
37             started =
38             SharingStarted.WhileSubscribed(5000),
39             initialValue =
40             settingsManager.isDarkModeEnabled()
41         )
42     init {
43         fetchMovies()
44     }
45     fun fetchMovies(fetchFromRemote: Boolean = true) {
46         viewModelScope.launch {
47             repository.getPopularMovies(fetchFromRemote).collect {
48                 result ->
49                 _movies.value = when (result) {
50                     is Resource.Success -> {
51                         val uiMovies = result.data?.map
52                         { movieEntity ->
53                             ListMovie(
54                                 id = movieEntity.id,
55                                 title =
56                                 movieEntity.title,
57                                 imageUrl = if
58                                 (!movieEntity.posterPath.isNullOrEmpty())
59                                 Constants.TMDB_IMAGE_BASE_URL + movieEntity.posterPath
60                                 else null,
61                                 releaseDate =
62                                 movieEntity.releaseDate,
63                                 description =
64                                 movieEntity.overview,
65                                 voteAverage =
66                                 movieEntity.voteAverage
67                             )
68                         } ?: emptyList()
69                     } Resource.Success(uiMovies)
70                     is Resource.Error ->
71                     Resource.Error(result.message ?: "An unexpected error
72                     occurred", result.data?.map { movieEntity ->

```

```

62         ListMovie(
63             id = movieEntity.id,
64             title = movieEntity.title,
65             imageUrl = if
(!movieEntity.posterPath.isNullOrEmpty())
Constants.TMDB_IMAGE_BASE_URL + movieEntity.posterPath
else null,
66             releaseDate =
movieEntity.releaseDate,
67             description =
movieEntity.overview,
68             voteAverage =
movieEntity.voteAverage
69         )
70     } ?: emptyList())
71     is Resource.Loading ->
Resource.Loading()
72     }
73 }
74 }
75 }
76
77 fun setDarkMode(isEnabled: Boolean) {
78     settingsManager.setDarkModeEnabled(isEnabled)
79 }
80
81 fun onImdbClicked(url: String) {
82     Timber.d("URL clicked: $url")
83     _navigateToUrl.value = url
84 }
85
86 fun onDetailClicked(movie: ListMovie) {
87     Timber.d("Detail clicked for movie:
${movie.title}")
88     _navigateToDetail.value = movie
89 }
90
91 fun onUrlNavigated() {
92     _navigateToUrl.value = null
93 }
94
95 fun onDetailNavigated() {
96     _navigateToDetail.value = null
97 }
98 }

```

viewModel/ModelViewModelFactory.kt*Tabel 17. Source Code MovieViewModelFactory.kt Compose*

1	package com.example.mlistcompose.viewModel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import
	com.example.mlistcompose.data.preferences.SettingsManager
6	import
	com.example.mlistcompose.repository.MovieRepository
7	
8	class MovieViewModelFactory(
9	private val repository: MovieRepository,
10	private val settingsManager: SettingsManager
11	) : ViewModelProvider.Factory {
12	override fun <T : ViewModel> create(modelClass:
	Class<T>): T {
13	if
	(modelClass.isAssignableFrom(MovieViewModel::class.java
	)) {
14	@Suppress("UNCHECKED_CAST")
15	return MovieViewModel(repository,
	settingsManager) as T
16	}
17	throw IllegalArgumentException("Unknown
	ViewModel class")
18	}
19	}

MListComposeApplication.kt*Tabel 18. Source Code MovieViewModelFactory.kt Compose*

1	package com.example.mlistcompose
2	
3	import android.app.Application
4	import com.example.mlistcompose.data.local.AppDatabase
5	import com.example.mlistcompose.network.RetrofitClient
6	import
	com.example.mlistcompose.repository.MovieRepository
7	import timber.log.Timber
8	import

	com.example.mlistcompose.viewmodel.MovieViewModelFactory
9	import
	com.example.mlistcompose.data.preferences.SettingsManager
10	
11	class MListComposeApplication : Application() {
12	
13	private val database: AppDatabase by lazy {
	AppDatabase.getDatabase(this) }
14	private val movieDao by lazy { database.movieDao() }
15	private val tmdbApiService by lazy {
	RetrofitClient.tmdbApiService }
16	private val settingsManager: SettingsManager by
	lazy { SettingsManager(this) }
17	
18	val movieRepository: MovieRepository by lazy {
19	MovieRepository(tmdbApiService, movieDao)
20	}
21	
22	val movieViewModelFactory: MovieViewModelFactory by
	lazy {
23	MovieViewModelFactory(movieRepository,
	settingsManager)
24	}
25	
26	override fun onCreate() {
27	super.onCreate()
28	if (BuildConfig.DEBUG) {
29	Timber.plant(Timber.DebugTree())
30	}
31	}
32	}

MainActivity.kt

Tabel 19. Source Code MainActivity.kt Compose

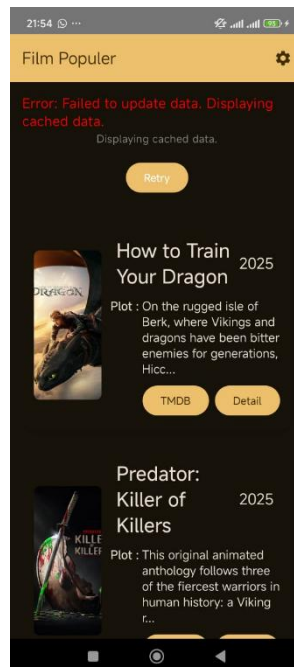
1	package com.example.mlistcompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.viewModels
7	import androidx.compose.foundation.layout.fillMaxSize
8	import androidx.compose.material3.MaterialTheme


```

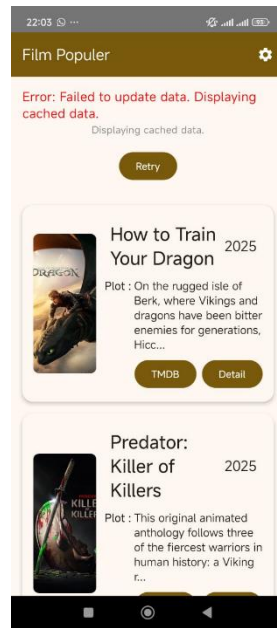
44         startDestination = "movieList"
45     ) {
46         composable("movieList") {
47             MovieListScreen(
48                 viewModel = viewModel,
49                 navController =
navController
50             )
51         }
52         composable("settings") {
53             SettingsScreen(navController = navController, viewModel
= viewModel)
54         }
55     }
56 }
57 }
58 }
59 }
60 }

```

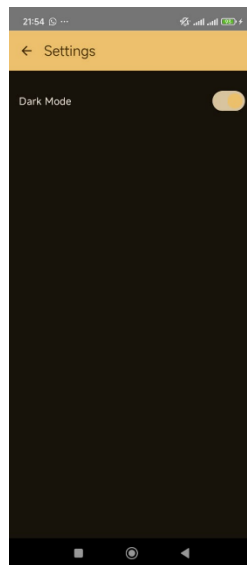
B. Output Program



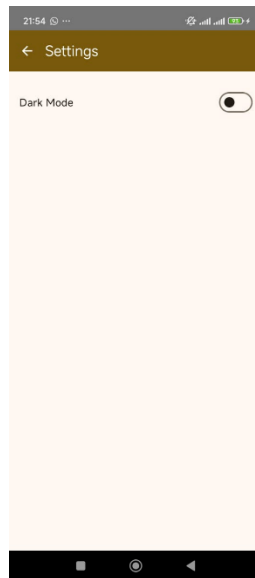
Gambar 1. Tampilan Aplikasi Ketika Offline



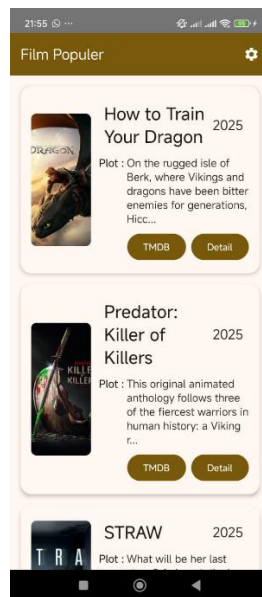
Gambar 2. Tampilan Aplikasi Ketika Offline dalam Light Mode



Gambar 3. Tampilan Tombol Ubah Dark/Light Mode



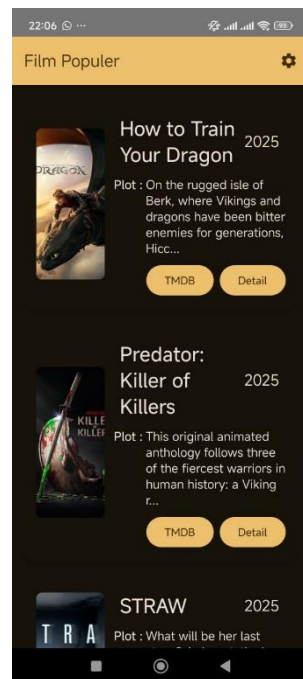
Gambar 4. Tampilan Tombol Ubah Dark/Light Mode



Gambar 5. Tampilan Aplikasi Saat Retry Ditekan dan Online dalam Light Mode



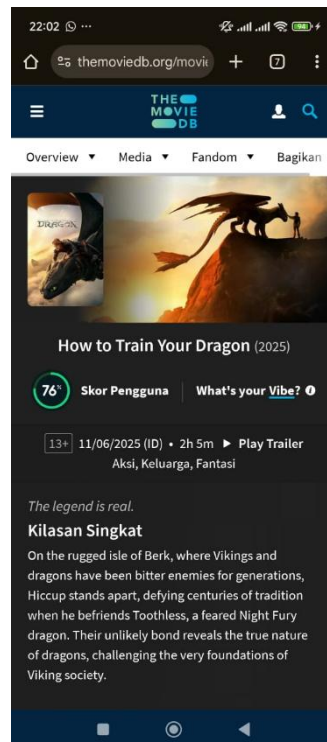
Gambar 6. Tombol Detail Ditekan dalam Light Mode



Gambar 7. Tampilan Aplikasi Saat Retry Ditekan dan Online dalam Dark Mode



Gambar 8. Tombol Detail Ditekan dalam Dark Mode



Gambar 9. Tombol TMDB Saat Ditekan

C. Pembahasan

Jetpack Compose

data/local/AppDatabase.kt

Pada baris [1–6], file ditempatkan dalam package `data.local` yang menunjukkan lokasi penyimpanan data lokal, lalu mengimpor `Context` untuk keperluan akses sistem, serta mengimpor komponen-komponen penting dari library Room seperti `@Database`, `Room`, dan `RoomDatabase` yang digunakan untuk membangun dan mengelola database lokal dalam aplikasi Android.

Pada baris [8–10], class `AppDatabase` dideklarasikan sebagai abstract class yang mewarisi `RoomDatabase`, dan diberi anotasi `@Database` untuk mendefinisikan entitas database yaitu `MovieEntity`, versi database (1), serta menonaktifkan ekspor skema; di dalamnya terdapat fungsi abstrak `movieDao()` yang digunakan Room untuk mengakses DAO (`MovieDao`) sebagai antarmuka operasi terhadap tabel film.

Pada baris [12–28], dideklarasikan objek companion object untuk menyediakan akses global ke instance database, di mana variabel `INSTANCE` bertipe nullable `AppDatabase` ditandai dengan anotasi `@Volatile` agar perubahan nilainya terlihat oleh semua thread; fungsi `getDatabase(context: Context)` akan mengembalikan instance database yang sudah ada atau membuat instance baru secara thread-safe menggunakan `synchronized`, dengan membangun database bernama `"movie_database"` menggunakan `Room.databaseBuilder`, lalu menyimpannya ke dalam `INSTANCE` agar dapat digunakan kembali.

data/local/MovieDao.kt

Pada baris [1–7], file ditempatkan dalam package `data.local` yang menandakan bahwa isi file ini berkaitan dengan data lokal, lalu mengimpor anotasi dan fungsi dari library Room seperti `@Dao`, `@Insert`, `OnConflictStrategy`, dan `@Query` untuk mendefinisikan operasi database, serta mengimpor `Flow` dari Kotlin Coroutines yang memungkinkan pemrosesan data secara asinkron dan berkelanjutan dari database.

Pada baris [9–19], dideklarasikan interface `MovieDao` yang ditandai dengan anotasi `@Dao` sebagai Data Access Object untuk Room, yang berisi tiga fungsi: `insertMovies()` untuk menyisipkan daftar film ke database dengan strategi konflik `REPLACE`, `getAllMovies()` untuk mengambil seluruh data film secara real-time dan terurut berdasarkan tanggal rilis secara menurun dalam bentuk `Flow`, serta `deleteAllMovies()` untuk menghapus seluruh data film dari tabel `movies`.

data/local/MovieEntity.kt

Pada baris [1–4], file ditempatkan dalam package `data.local` sebagai bagian dari struktur penyimpanan data lokal, lalu mengimpor anotasi `@Entity` untuk mendefinisikan class sebagai entitas tabel dalam Room, serta `@PrimaryKey` untuk menandai properti sebagai kunci utama tabel.

Pada baris [6–14], dideklarasikan data class `MovieEntity` yang ditandai dengan anotasi `@Entity(tableName = "movies")` sebagai representasi tabel `movies` dalam database `Room`, dengan properti `id` sebagai `@PrimaryKey`, serta properti lainnya seperti `title`, `posterPath`, `releaseDate`, `overview`, dan `voteAverage` yang masing-masing merepresentasikan kolom-kolom dalam tabel dan menyimpan informasi detail mengenai film.

`data/local/preferences/SettingManager.kt`

Pada baris [1–8], file ditempatkan dalam package `data.preferences` yang menunjukkan bahwa isinya berkaitan dengan pengelolaan preferensi pengguna, lalu mengimpor `Context` dan `SharedPreferences` untuk mengakses dan menyimpan data preferensi, edit dari `androidx.core.content` untuk mempermudah proses penyimpanan data, serta `Flow`, `callbackFlow`, dan `awaitClose` dari Kotlin Coroutines untuk memungkinkan pemantauan perubahan data preferensi secara asinkron dan reaktif.

Pada baris [10–11], dideklarasikan class `SettingsManager` yang menerima `Context` sebagai parameter untuk mengakses sistem Android, lalu membuat objek `prefs` bertipe `SharedPreferences` dengan nama `"app_settings"` dan mode privat (`Context.MODE_PRIVATE`) agar data hanya dapat diakses oleh aplikasi itu sendiri, digunakan untuk menyimpan dan mengambil pengaturan aplikasi secara permanen.

Pada baris [13–15], dideklarasikan companion object yang berisi konstanta `KEY_DARK_MODE` untuk menyimpan nama kunci (`"dark_mode"`) yang digunakan dalam `SharedPreferences`, agar konsisten saat menyimpan atau mengambil nilai terkait pengaturan mode gelap aplikasi.

Pada baris [17–26], fungsi `getDarkModeEnabledAsFlow()` mengembalikan `Flow<Boolean>` menggunakan `callbackFlow`, yang memungkinkan pengamatan perubahan nilai dari `SharedPreferences`; di dalamnya dibuat listener untuk mendeteksi perubahan pada kunci `KEY_DARK_MODE` dan mengirim nilai terbaru melalui `trySend()`, lalu listener tersebut didaftarkan ke `prefs`, nilai awal dikirim segera setelah fungsi dipanggil, dan `awaitClose` digunakan untuk mencabut listener saat aliran tidak lagi digunakan, memastikan pembersihan sumber daya yang aman.

Pada baris [28–30], fungsi `isDarkModeEnabled()` mengembalikan nilai `Boolean` yang diambil dari `SharedPreferences` dengan kunci `KEY_DARK_MODE`, dan akan menghasilkan `false` secara default jika belum ada nilai yang disimpan, berfungsi untuk mengecek apakah mode gelap diaktifkan oleh pengguna.

Pada baris [32–34], fungsi `setDarkModeEnabled(isEnabled: Boolean)` digunakan untuk menyimpan nilai mode gelap ke dalam `SharedPreferences` dengan memanfaatkan ekstensi `edit` agar lebih ringkas, di mana nilai `isEnabled` disimpan menggunakan kunci `KEY_DARK_MODE` untuk mencatat apakah mode gelap diaktifkan atau tidak.

model/network/MovieApi.kt

Pada baris [1–4], file ditempatkan dalam package `model.network` yang menunjukkan bahwa isinya berkaitan dengan model data dari jaringan, lalu mengimpor anotasi `@SerializedName` dan `@Serializable` dari Kotlin Serialization yang digunakan untuk memetakan dan mengatur serialisasi/deserialisasi objek data dari dan ke format JSON saat berkomunikasi dengan API.

Pada baris [6–14], dideklarasikan data class `MovieApi` yang ditandai dengan anotasi `@Serializable` agar dapat diserialisasi dan dideserialisasi secara otomatis saat komunikasi dengan API, dengan properti seperti `id`, `title`, `posterPath`, `releaseDate`, `overview`, dan `voteAverage`, di mana beberapa properti menggunakan anotasi `@SerializedName` untuk memetakan nama field JSON (`poster_path`, `release_date`, `vote_average`) ke nama variabel Kotlin yang sesuai.

model/network/MovieResponse.kt

Pada baris [1–4], file ditempatkan dalam package `model.network` yang menandakan bahwa isinya merupakan model data dari jaringan (`network`), lalu mengimpor anotasi `@SerializedName` dan `@Serializable` dari Kotlin Serialization untuk memungkinkan objek data disesuaikan dengan format JSON saat proses serialisasi dan deserialisasi data dari API.

Pada baris [6–12], dideklarasikan data class `MovieResponse` yang ditandai dengan anotasi `@Serializable` untuk memungkinkan serialisasi dan deserialisasi data dari API, dengan properti `page` sebagai nomor halaman saat ini, `results` sebagai daftar objek `MovieApi`, serta `totalPages` dan `totalResults` yang masing-masing dipetakan dari nama field JSON `"total_pages"` dan `"total_results"` menggunakan anotasi `@SerializedName`.

model/ListMovie.kt

Pada baris [1–10], file ditempatkan dalam package `model` yang menunjukkan bahwa class ini merupakan bagian dari model data dalam aplikasi, lalu dideklarasikan data class `ListMovie` yang merepresentasikan struktur data film untuk digunakan di UI, dengan properti `id`, `title`, `imageUrl`, `releaseDate`, `description`, dan `voteAverage` yang masing-masing menyimpan informasi dasar mengenai sebuah film dalam bentuk yang lebih ringan dan terformat untuk ditampilkan.

network/RetrofitClient.kt

Pada baris [1–11], file ditempatkan dalam package `network` yang menunjukkan bahwa isinya berkaitan dengan konfigurasi jaringan, lalu mengimpor `Constants` untuk mengambil nilai konstan seperti base URL atau API key, converter dari Kotlin Serialization untuk Retrofit, objek `Json` untuk parsing JSON, serta komponen dari `OkHttp` seperti `Interceptor`, `OkHttpClient`, `MediaType`, dan `HttpLoggingInterceptor`, dan terakhir Retrofit sebagai library utama untuk melakukan komunikasi HTTP dengan API.

Pada baris [13], dideklarasikan object `RetrofitClient` sebagai singleton, yaitu objek tunggal yang digunakan untuk mengelola dan menyediakan instance Retrofit secara global di seluruh aplikasi, sehingga konfigurasi client HTTP tidak perlu dibuat berulang kali.

Pada baris [15], dibuat objek json dari Json dengan konfigurasi `ignoreUnknownKeys = true`, yang berarti saat melakukan deserialisasi dari JSON, properti yang tidak dikenali (tidak ada dalam data class) akan diabaikan agar tidak menyebabkan error, sehingga aplikasi tetap stabil meskipun respons API mengandung field tambahan yang tidak dipakai.

Pada baris [17–19], dibuat `loggingInterceptor` menggunakan `HttpLoggingInterceptor` dan disetel levelnya ke BODY, yang berarti semua detail permintaan dan respons HTTP—including header, body, dan metadata—akan dicatat ke log, berguna untuk debugging dan memantau lalu lintas jaringan selama pengembangan.

Pada baris [21–28], dibuat `authInterceptor` sebagai `Interceptor` kustom yang secara otomatis menambahkan parameter query `api_key` dengan nilai dari `Constants.TMDB_API_KEY` ke setiap permintaan jaringan, dengan mengambil `originalRequest`, membangun ulang URL-nya menggunakan `newBuilder()`, lalu membuat `newRequest` dengan URL baru tersebut, dan akhirnya meneruskannya (`proceed`) ke jaringan — ini mempermudah otentikasi otomatis ke API tanpa perlu menambahkan API key secara manual di setiap endpoint.

Pada baris [30–33], dibuat instance `okHttpClient` menggunakan `OkHttpClient.Builder()` dengan menambahkan dua `interceptor` yaitu `authInterceptor` untuk menyisipkan API key secara otomatis ke setiap request, dan `loggingInterceptor` untuk mencatat detail permintaan dan respons HTTP, lalu semuanya dibangun menjadi satu objek `OkHttpClient` yang akan digunakan oleh Retrofit untuk menjalankan komunikasi jaringan.

Pada baris [35–43], properti `tmdbApiService` dideklarasikan dengan anotasi `@OptIn(ExperimentalSerializationApi::class)` karena menggunakan fitur eksperimental dari Kotlin Serialization, dan dibuat secara lazy agar hanya diinisialisasi saat pertama kali diakses; di dalamnya dibuat instance Retrofit dengan `baseUrl` dari `Constants.TMDB_BASE_URL`, menggunakan `okHttpClient` yang sudah dikonfigurasi sebelumnya, serta menambahkan `converter factory` dari json untuk memproses konten bertipe "application/json", lalu membangun dan menghasilkan implementasi dari `TmdbApiService` yang siap digunakan untuk mengakses API TMDB.

network/TmdbApiService.kt

Pada baris [1–5], file ditempatkan dalam package `network` yang menunjukkan bahwa isinya berkaitan dengan komunikasi API, lalu mengimpor `MovieResponse` sebagai model respons dari server, serta anotasi `@GET` dan `@Query` dari Retrofit yang digunakan untuk mendefinisikan metode HTTP dan parameter query dalam interface layanan API.

Pada baris [7–13], dideklarasikan interface `TmdbApiService` yang merepresentasikan layanan API Retrofit, dengan satu fungsi `getPopularMovies()` yang ditandai dengan anotasi `@GET("movie/popular")` untuk mengambil data film populer dari endpoint tersebut, lalu menggunakan anotasi `@Query` untuk menyisipkan parameter `language` dan `page` dalam URL permintaan secara default, dan fungsi ini bersifat suspend agar dapat dipanggil dalam coroutine, serta mengembalikan objek `MovieResponse` sebagai hasil dari respons API.

repository/MovieRepository.kt

Pada baris [1–11], file ditempatkan dalam package repository yang menunjukkan bahwa fungsinya sebagai penghubung antara data dari jaringan dan database lokal, lalu mengimpor berbagai komponen seperti MovieDao dan MovieEntity dari layer lokal, MovieApi sebagai model data dari API, TmdbApiService sebagai interface untuk akses ke API TMDB, dan Resource sebagai wrapper status data (loading, success, error), serta Flow, flow, dan firstOrNull dari coroutine untuk pemrosesan data secara asinkron, dan Timber untuk logging yang membantu dalam pencatatan error atau debugging.

Pada baris [13–16], dideklarasikan kelas MovieRepository yang memiliki dua parameter konstruktor: apiService untuk melakukan permintaan data ke API TMDB, dan movieDao untuk berinteraksi dengan database lokal Room, sehingga repository ini bertugas mengelola logika pengambilan dan penyimpanan data dari kedua sumber tersebut.

Pada baris [17–20], fungsi getPopularMovies didefinisikan untuk mengembalikan aliran data (Flow) yang dibungkus dalam Resource agar statusnya dapat merepresentasikan loading, success, atau error, dengan parameter fetchFromRemote untuk menentukan apakah perlu mengambil data terbaru dari API; fungsi ini langsung memancarkan status Resource.Loading() sebagai indikator bahwa proses pengambilan data sedang berlangsung.

Pada baris [22–26], data film diambil terlebih dahulu dari database lokal menggunakan movieDao.getAllMovies().firstOrNull(), lalu jika hasilnya tidak kosong (isNullOrEmpty() bernilai false), data tersebut langsung dipancarkan sebagai Resource.Success, menandakan data berhasil dimuat dari cache, dan juga dicatat jumlah datanya ke log menggunakan Timber untuk keperluan debugging.

Pada baris [28], variabel shouldLoadFromApi digunakan untuk menentukan apakah perlu mengambil data dari API, dengan logika bahwa pengambilan dilakukan jika fetchFromRemote bernilai true (dipaksa ambil dari API) atau jika localMovies kosong/null, sehingga memastikan data selalu tersedia dan diperbarui jika dibutuhkan.

Pada baris [30–46], jika shouldLoadFromApi bernilai true, maka dilakukan proses pengambilan data film dari API dalam blok try, dimulai dengan logging proses melalui Timber, lalu memanggil apiService.getPopularMovies() untuk mendapatkan respons film populer yang hasilnya diubah menjadi MovieEntity melalui map dan fungsi ekstensi toMovieEntity(), setelah itu database lokal dikosongkan (deleteAllMovies()) dan diisi ulang dengan data baru (insertMovies()), kemudian log jumlah data yang berhasil di-cache, dan sebagai langkah akhir, data terbaru dari database kembali diambil untuk dipancarkan sebagai Resource.Success, atau jika gagal, dipancarkan sebagai Resource.Error dengan pesan kegagalan mengambil cache.

Pada baris [48–55], blok catch menangani jika terjadi exception selama proses pengambilan data dari API, dengan mencatat error melalui Timber.e, lalu memeriksa apakah localMovies kosong; jika iya, maka dipancarkan Resource.Error dengan pesan bahwa server tidak dapat dijangkau dan mengirimkan list kosong, sedangkan jika localMovies masih tersedia, maka

tetap dipancarkan `Resource.Error` tetapi dengan data cache yang ada, sehingga pengguna tetap dapat melihat data meskipun update gagal.

Pada baris [56–61], blok `else` dijalankan jika `shouldLoadFromApi` bernilai `false` (artinya tidak mengambil data dari API), lalu dilakukan pengecekan kembali apakah `localMovies` kosong; jika iya, maka dipancarkan `Resource.Error` dengan pesan bahwa tidak ada data cache yang tersedia dan pengambilan dari remote juga tidak dilakukan, sehingga kondisi ini menangani skenario terburuk saat data tidak bisa dimuat dari manapun.

Pada baris [63–72], didefinisikan fungsi ekstensi `toMovieEntity()` yang mengubah objek `MovieApi` menjadi `MovieEntity`, dengan mengisi masing-masing properti dari data API ke entitas lokal, termasuk memberi nilai default "No Title" jika judul (title) bernilai `null`, sehingga fungsi ini memastikan data yang diterima dari jaringan dapat langsung disimpan ke database lokal Room dalam format yang sesuai.

screen/DetailActivity

Pada baris [1–21], file ini berada di package `screen` dan mengimpor berbagai komponen yang dibutuhkan untuk membuat tampilan layar detail menggunakan Jetpack Compose, termasuk `ComponentActivity`, fungsi layout seperti `Column`, `Box`, dan `Spacer`, komponen Material3 seperti `Text` dan `Surface`, fungsi scroll, pengaturan gaya teks (`TextStyle`, `FontWeight`, `sp`), serta library `Coil` (`AsyncImage`) untuk memuat gambar secara asinkron, dan juga `SettingsManager` untuk pengaturan preferensi, semua ini disiapkan dalam konteks penggunaan tema `MListComposeTheme`.

Pada baris [23–25], kelas `DetailActivity` diturunkan dari `ComponentActivity` dan mengoverride fungsi `onCreate`, yang digunakan untuk menginisialisasi aktivitas ketika pertama kali dibuat, serta memanggil `super.onCreate(savedInstanceState)` agar logika inisialisasi bawaan tetap dijalankan.

Pada baris [27–34], data intent seperti `title`, `imageUrl`, `releaseDate`, `description`, dan `voteAverage` diambil dari parameter yang dikirim dari aktivitas sebelumnya dengan nilai default jika `null`, lalu `SettingsManager` diinisialisasi untuk membaca preferensi dark mode dan menentukan apakah mode gelap aktif melalui `isDarkModeEnabled`.

Pada baris [36–41], `setContent` digunakan untuk menetapkan UI Compose dengan tema `MListComposeTheme` yang disesuaikan berdasarkan status dark mode, lalu sebuah `Surface` dibuat untuk mengisi seluruh layar (`fillMaxSize`) dan menerapkan warna latar belakang sesuai skema warna tema aktif.

Pada baris [42–47], digunakan `Column` sebagai layout vertikal yang mengisi lebar penuh layar (`fillMaxWidth()`), diberi padding 16dp di sekelilingnya, dan diatur agar bisa discroll secara vertikal dengan `verticalScroll(rememberScrollState())`, memungkinkan konten panjang tetap dapat ditampilkan dengan baik.

Pada baris [48–57], komponen `AsyncImage` digunakan untuk menampilkan poster film yang dimuat dari URL `imageUrl`, dengan deskripsi gambar berupa judul `title`. Gambar diatur

memiliki tinggi 300dp, lebar memenuhi layar, dan ditempatkan di tengah secara horizontal. Skala konten diatur ke Fit agar seluruh gambar tampil tanpa terpotong. Setelah gambar, Spacer ditambahkan untuk memberi jarak vertikal sebesar 16dp sebelum elemen UI berikutnya.

Pada baris [59–99], beberapa komponen Text digunakan untuk menampilkan detail informasi film. Pertama, judul film (title) ditampilkan dengan ukuran huruf besar (36sp), tebal (Bold), dan warna mengikuti tema (onBackground). Kemudian, tanggal rilis (releaseDate) dan rating film (voteAverage) ditampilkan dengan ukuran font sedang (18sp), serta diberi padding bawah untuk pemisahan visual. Setelah itu, label "Plot :" dicetak dengan ukuran 20sp dan huruf tebal sebagai penanda deskripsi film. Spacer dengan tinggi 8dp ditambahkan untuk memberi jarak sebelum deskripsi (overview), yang ditampilkan dalam font 16sp dan warna tema. Semua teks mengikuti skema warna dari tema agar responsif terhadap mode gelap/terang.

screen/MovieListScreen.kt

Pada baris [1–53], sejumlah *import* seperti LazyColumn, AsyncImage, Scaffold, TopAppBar, MovieViewModel, Resource, dan Intent digunakan untuk membangun tampilan daftar film, memuat gambar, menangani navigasi, serta menghubungkan data dari ViewModel ke UI dengan komponen Material 3.

Pada baris [55–60], fungsi @Composable bernama MovieListScreen dideklarasikan dengan anotasi @OptIn(ExperimentalMaterial3Api::class, ExperimentalMaterial3Api::class) untuk mengaktifkan fitur eksperimental dari Material 3, dan menerima parameter viewModel bertipe MovieViewModel serta NavController untuk navigasi antar layar.

Pada baris [61–64], beberapa state Compose diobservasi: moviesResource memantau daftar film dari viewModel.movies, navigateToUrl dan navigateToDetail memantau event navigasi dari ViewModel, semuanya menggunakan collectAsState() untuk reaktifitas UI, serta context diambil dari LocalContext.current untuk akses konteks Android saat ini.

Pada baris [66–70], jika navigateToUrl tidak null, kode membuat Intent untuk membuka URL di browser menggunakan Intent.ACTION_VIEW dan toUri(), lalu memulai aktivitas tersebut melalui context.startActivity(intent), setelahnya memanggil viewModel.onUrlNavigated() untuk menandai bahwa navigasi URL telah diproses agar event tidak di-trigger ulang.

Pada baris [72–83], jika navigateToDetail tidak null, kode membuat Intent untuk berpindah ke DetailActivity dengan menyisipkan data film seperti id, title, imageUrl, releaseDate, description, dan voteAverage menggunakan putExtra(). Setelah itu, intent dijalankan dengan context.startActivity(intent), dan viewModel.onDetailNavigated() dipanggil untuk menandai bahwa navigasi ke detail telah selesai agar event tidak ter-trigger ulang.

Pada baris [85–122], Scaffold digunakan untuk membuat layout utama dengan TopAppBar berjudul "Film Populer" dan tombol pengaturan yang menavigasi ke layar "settings" saat ditekan. Warna latar dan teks mengikuti skema tema. Di dalam konten Scaffold, sebuah

Column dengan padding responsif dari Scaffold menampilkan UI berdasarkan status `moviesResource`: saat status `Loading`, ditampilkan `CircularProgressIndicator` dan teks "Loading movies..." di tengah layar dengan warna putih untuk memberi tahu pengguna bahwa data film sedang dimuat.

Pada baris [123–137], ketika `moviesResource` berstatus `Success`, data film yang berhasil diambil (`movieList`) ditampilkan menggunakan `LazyColumn` dengan padding 16dp dan jarak antar item 16dp. Setiap item dalam daftar di-render dengan komponen `MovieItems` yang menerima data film, serta dua callback untuk aksi klik: membuka detail film (`onDetailClicked`) dan membuka URL TMDb (`onTmdbClicked`), keduanya memanggil fungsi di `viewModel` sesuai aksi pengguna.

Pada baris [138–178], saat `moviesResource` berstatus `Error`, pesan error disimpan dalam `errorMessage` dan data cache dalam `cachedData`. Jika data cache tersedia, daftar film tetap ditampilkan menggunakan `LazyColumn` dengan padding dan jarak antar item, dan di atas daftar ditampilkan pesan error berwarna merah, keterangan bahwa data cache sedang ditampilkan berwarna abu-abu, serta tombol "Retry" yang memicu pengambilan ulang data dari API lewat `viewModel.fetchMovies(true)`. Setiap item film di-render dengan `MovieItems` dan callback untuk membuka detail atau URL TMDb seperti sebelumnya.

Pada baris [179–190], jika status error terjadi dan tidak ada data cache, maka ditampilkan tampilan penuh yang berisi pesan error berwarna merah dan tombol "Retry" berwarna putih di tengah layar, yang ketika ditekan akan memicu pengambilan ulang data film melalui `viewModel.fetchMovies(true)`.

Pada baris [198–203], didefinisikan fungsi `composable MovieItems` yang menerima parameter `movie` bertipe `ListMovie`, serta dua callback yaitu `onDetailClicked` tanpa argumen yang dipanggil saat item detail diklik, dan `onTmdbClicked` dengan argumen `String` yang dipanggil saat tautan TMDb diklik. Fungsi ini bertugas menampilkan item film dalam daftar dengan interaksi yang sesuai.

Pada baris [204–298], komponen `MovieItems` membangun kartu (`Card`) dengan sudut membulat dan elevasi ringan, yang berisi tata letak `Row` untuk menampilkan gambar film (`AsyncImage`) di sisi kiri dengan ukuran tetap dan sudut membulat, serta kolom di kanan yang berisi judul film (besar dan tebal), tanggal rilis (diambil dari substring sebelum tanda "-" atau "N/A"), dan ringkasan plot yang dipotong hingga 100 karakter dengan tanda ellipsis. Di bagian bawah, ada dua tombol: satu untuk membuka URL TMDb berdasarkan ID film yang memanggil `onTmdbClicked` dengan URL lengkap, dan satu lagi untuk melihat detail yang memanggil `onDetailClicked`. Semua elemen teks dan tombol menggunakan warna dan gaya yang konsisten dengan tema aplikasi agar responsif terhadap mode gelap/terang.

screen/SettingScreen.kt

Pada baris [1–10], kode mengimpor paket dan library yang diperlukan untuk membuat UI dengan Jetpack Compose, termasuk layout dasar (`Column`, `Row`, `Spacer`, `Modifier`), ikon `ArrowBack` untuk navigasi, komponen Material3 seperti `TopAppBar`, `Button`, dan `Text`, serta

fungsi pendukung Compose seperti `@Composable` dan state management (`collectAsState`, `getValue`). Selain itu, `NavController` diimpor untuk mengelola navigasi antar layar, dan `MovieViewModel` untuk mengakses data film dari layer `ViewModel`.

Pada baris [16–21], fungsi composable `SettingsScreen` didefinisikan dengan anotasi `@OptIn` untuk menggunakan API eksperimental `Material3`, menerima `navController` untuk navigasi layar dan `viewModel` untuk mengakses data atau logika terkait film, sebagai parameter input.

Pada baris [22–79], state `isDarkMode` diambil dari `viewModel` menggunakan `collectAsState` untuk memantau status mode gelap secara reaktif, lalu `Scaffold` digunakan untuk membangun tampilan dengan `TopAppBar` berjudul "Settings" yang memiliki tombol navigasi kembali; di dalam konten utama terdapat `Column` yang berisi `Row` dengan teks label "Dark Mode" dan `Switch` yang mencerminkan status `isDarkMode` serta memungkinkan pengguna mengubah mode gelap yang kemudian diteruskan ke `viewModel.setDarkMode` untuk memperbarui pengaturan, dengan warna `switch` yang menyesuaikan tema `Material3` agar konsisten dengan mode terang atau gelap.

utils/Constants.kt

Pada baris [1–8], didefinisikan objek `Constants` yang berisi konstanta API untuk `TMDB`, termasuk `TMDB_API_KEY` untuk autentikasi, URL dasar API (`TMDB_BASE_URL`), URL dasar untuk mengambil gambar film (`TMDB_IMAGE_BASE_URL`), dan URL dasar untuk mengakses halaman web detail film di `TMDB` (`TMDB_MOVIE_WEB_BASE_URL`).

utils/resource.kt

Pada baris [1–7], dibuat kelas sealed `Resource` generik untuk merepresentasikan status data yang diambil, dengan tiga subclass: `Success` untuk data berhasil dengan isi data, `Error` untuk kondisi gagal dengan pesan error dan opsional data, serta `Loading` untuk status pemuatan yang bisa membawa data sementara.

viewmodel/MovieViewModel.kt

Pada baris [1–17], dilakukan import berbagai kelas dan pustaka yang diperlukan dalam `MovieViewModel`, termasuk kelas `ViewModel` dan `viewModelScope` dari `Android Lifecycle`, kelas dan utilitas terkait flow dari `Kotlin Coroutines`, model data `ListMovie`, repository `MovieRepository`, manajer preferensi `SettingsManager`, konstanta aplikasi, kelas `Resource` untuk pengelolaan status data, serta pustaka logging `Timber` untuk pencatatan log.

Pada baris [19–22], dideklarasikan kelas `MovieViewModel` yang merupakan subclass dari `ViewModel`. Kelas ini menerima dua dependency melalui konstruktor: repository bertipe `MovieRepository` untuk mengelola data film, dan `settingsManager` bertipe `SettingsManager` untuk mengelola preferensi aplikasi seperti pengaturan tema.

Pada baris [23–24], dibuat variabel private bernama `_movies` dengan tipe `MutableStateFlow` yang menyimpan objek `Resource` berisi daftar `ListMovie`, diinisialisasi dengan status `Resource.Loading()` untuk menandakan data sedang dimuat; lalu variabel publik `movies` bertipe `StateFlow` yang hanya dapat dibaca, mengambil nilai dari `_movies` menggunakan

fungsi `asStateFlow()` agar UI dapat mengamati perubahan data tanpa bisa memodifikasinya langsung.

Pada baris [26–27], dibuat variabel `private _navigateToUrl` berupa `MutableStateFlow` yang menyimpan URL bertipe `String?` dan diinisialisasi `null` (belum ada URL); lalu variabel publik `navigateToUrl` bertipe `StateFlow<String?>` yang hanya dapat dibaca, mengambil nilai dari `_navigateToUrl` dengan `asStateFlow()` agar UI bisa mengamati perubahan URL untuk navigasi tanpa bisa mengubahnya langsung.

Pada baris [29–30], dibuat variabel `private _navigateToDetail` berupa `MutableStateFlow` yang menyimpan objek `ListMovie?` dengan nilai awal `null`, serta variabel publik `navigateToDetail` berupa `StateFlow<ListMovie?>` yang hanya bisa dibaca dan meng-expose data dari `_navigateToDetail` menggunakan `asStateFlow()`, memungkinkan UI mengamati perubahan detail film untuk navigasi ke halaman detail tanpa dapat memodifikasi langsung.

Pada baris [32–38], `isDarkModeEnabled` adalah `StateFlow<Boolean>` yang mengamati status mode gelap dari `settingsManager` secara reaktif; data diambil dari `getDarkModeEnabledAsFlow()`, di-filter agar hanya update unik dengan `distinctUntilChanged()`, dan diubah menjadi `StateFlow` yang aktif selama 5 detik setelah subscriber terakhir berhenti dengan `stateIn()` menggunakan `viewModelScope`, serta nilai awal diambil dari `settingsManager.isDarkModeEnabled()`.

Pada baris [40–42], blok `init` berfungsi sebagai inisialisasi awal `ViewModel` dengan langsung memanggil fungsi `fetchMovies()` untuk mulai mengambil data film ketika `ViewModel` dibuat.

Pada baris [44–75], `fetchMovies` adalah fungsi di dalam `ViewModel` yang memanggil `repository.getPopularMovies(fetchFromRemote)` dalam `coroutine viewModelScope.launch`, lalu mengumpulkan hasilnya secara reaktif dengan `collect`; hasil `Resource` kemudian dipetakan ke `_movies.value` sesuai dengan statusnya: jika `Success`, data film diubah ke bentuk `ListMovie` dan dikemas kembali dalam `Resource.Success`; jika `Error`, pesan error disertakan dan data juga dipetakan ke `ListMovie` bila tersedia; dan jika `Loading`, nilai `Resource.Loading()` langsung diberikan ke `_movies` untuk merepresentasikan status pemuatan.

Pada baris [77–79], `setDarkMode` adalah fungsi sederhana yang menerima parameter `isEnabled` bertipe `Boolean` untuk mengatur status mode gelap; fungsi ini meneruskan nilai tersebut ke `settingsManager.setDarkModeEnabled(isEnabled)` sehingga preferensi mode gelap disimpan secara persisten melalui `settingsManager`.

Pada baris [81–84], `onImdbClicked` adalah fungsi yang dipanggil saat pengguna mengklik tautan IMDb; fungsi ini mencatat URL yang diklik menggunakan `Timber.d` untuk keperluan debugging, lalu menetapkan nilai URL tersebut ke `_navigateToUrl.value` agar dapat diamati oleh UI dan memicu navigasi ke tautan tersebut secara reaktif.

Pada baris [86–89], `onDetailClicked` adalah fungsi yang dijalankan saat pengguna memilih salah satu film untuk melihat detailnya; fungsi ini mencatat judul film yang diklik

menggunakan Timber.d sebagai log debug, lalu menetapkan objek movie ke `_navigateToDetail.value` agar dapat diamati oleh UI dan memicu navigasi ke halaman detail film secara reaktif.

Pada baris [91–93], `onUrlNavigated` adalah fungsi yang dipanggil setelah navigasi ke URL selesai dilakukan; fungsi ini menetapkan nilai `_navigateToUrl.value` menjadi null untuk mereset event navigasi, sehingga tidak terjadi navigasi ulang saat nilai LiveData atau StateFlow diamati kembali oleh UI.

Pada baris [95–97], `onDetailNavigated` adalah fungsi yang dijalankan setelah navigasi ke halaman detail selesai; fungsi ini menetapkan `_navigateToDetail.value` menjadi null untuk mereset event navigasi, sehingga mencegah pemicu ulang navigasi saat observer UI membaca nilai tersebut kembali.

viewModel/ViewModelModelFactory.kt

Pada baris [1–6], paket `com.example.mlistcompose.viewModel` dideklarasikan sebagai lokasi kelas `ViewModel`; kemudian beberapa impor dilakukan: `ViewModel` dan `ViewModelProvider` dari `androidx.lifecycle` untuk mendukung arsitektur MVVM; `SettingsManager` dari `data.preferences` untuk mengelola preferensi aplikasi seperti mode gelap; serta `MovieRepository` dari `repository` yang digunakan untuk mengakses data film dari sumber data yang disediakan.

Pada baris [8–19], `MovieViewModelFactory` adalah kelas turunan dari `ViewModelProvider.Factory` yang digunakan untuk membuat instance `MovieViewModel` dengan dependency injection; konstruktor menerima dua parameter yaitu `repository` dan `settingsManager` yang akan diteruskan ke `MovieViewModel`; dalam override fungsi `create`, dilakukan pengecekan apakah `modelClass` adalah turunan dari `MovieViewModel`, dan jika ya maka objek `MovieViewModel` dikembalikan setelah dilakukan cast dengan penekanan peringatan `@Suppress("UNCHECKED_CAST")`; jika tidak cocok, maka `IllegalArgumentException` dilempar dengan pesan bahwa kelas `ViewModel` tidak dikenal.

MListComposeApplication.kt

Pada baris [1–9], paket `com.example.mlistcompose` dideklarasikan sebagai ruang lingkup utama aplikasi; beberapa impor dilakukan untuk mendukung inisialisasi dan dependency injection pada level aplikasi, termasuk `Application` dari `Android` sebagai superclass untuk membuat custom application class, `AppDatabase` untuk akses database lokal, `RetrofitClient` untuk komunikasi jaringan, `MovieRepository` sebagai pengelola logika data, `Timber` sebagai alat logging, `MovieViewModelFactory` untuk membuat instance `ViewModel` dengan dependensi, serta `SettingsManager` untuk pengelolaan preferensi pengguna seperti mode gelap.

Pada baris [11–16], `MListComposeApplication` adalah kelas turunan dari `Application` yang digunakan untuk mengelola inisialisasi global saat aplikasi pertama kali dijalankan; properti database diinisialisasi secara lazy untuk mendapatkan instance `AppDatabase`; `movieDao` diambil dari database sebagai akses objek DAO ke tabel film; `tmdbApiService` adalah service

Retrofit yang disediakan oleh RetrofitClient untuk melakukan permintaan API TMDb; dan settingsManager diinisialisasi dengan konteks aplikasi untuk mengelola preferensi seperti mode gelap secara persisten.

Pada baris [18–20], movieRepository adalah properti publik bertipe MovieRepository yang diinisialisasi secara lazy; objek ini dibuat dengan menyuntikkan dependensi tmdbApiService untuk mengambil data dari API TMDb dan movieDao untuk akses database lokal, sehingga movieRepository dapat bertindak sebagai sumber data terpusat yang menggabungkan data dari jaringan dan lokal.

Pada baris [22–24], movieViewModelFactory adalah properti publik bertipe MovieViewModelFactory yang diinisialisasi secara lazy; objek ini dibuat dengan menyuntikkan movieRepository dan settingsManager sebagai dependensi, sehingga movieViewModelFactory dapat digunakan untuk menghasilkan instance MovieViewModel dengan konstruktor kustom yang memerlukan parameter tersebut, sesuai dengan pola dependency injection dalam arsitektur MVVM.

Pada baris [26–31], onCreate adalah fungsi override dari kelas Application yang dipanggil saat aplikasi pertama kali dimulai; di dalamnya dipanggil super.onCreate() untuk memastikan inisialisasi bawaan tetap dijalankan; kemudian dilakukan pengecekan terhadap BuildConfig.DEBUG untuk mengetahui apakah aplikasi sedang dalam mode debug, dan jika ya maka Timber.plant(Timber.DebugTree()) dipanggil untuk menanam pohon debug Timber, memungkinkan pencatatan log secara detail selama pengembangan.

MainActivity.kt

Pada baris [1–20], paket com.example.mlistcompose dideklarasikan sebagai namespace utama aplikasi; berbagai impor disertakan untuk membangun tampilan utama aplikasi berbasis Jetpack Compose, termasuk ComponentActivity sebagai base activity, setContent untuk mengisi UI dengan komposisi, serta viewModels untuk inisialisasi ViewModel; elemen-elemen Compose seperti Modifier, MaterialTheme, Surface, collectAsState, dan getValue digunakan untuk pengaturan tema dan state UI; navigasi ditangani dengan NavHost, composable, dan rememberNavController; MovieListScreen dan SettingsScreen adalah tampilan yang dapat dinavigasi; MovieViewModel digunakan untuk mengelola logika data; SettingsManager dikelola untuk preferensi aplikasi; dan MListComposeTheme digunakan sebagai tema kustom aplikasi.

Pada baris [22–30], MainActivity adalah kelas turunan dari ComponentActivity yang berfungsi sebagai activity utama aplikasi; dalam fungsi onCreate, dipanggil super.onCreate(savedInstanceState) untuk menjalankan inisialisasi bawaan; viewModel diinisialisasi menggunakan delegasi by viewModels dengan movieViewModelFactory yang diambil dari MListComposeApplication untuk memastikan ViewModel dibuat dengan dependensi yang benar; kemudian settingsManager diinisialisasi menggunakan konteks activity (this) untuk memungkinkan akses preferensi lokal seperti status mode gelap.

Pada baris [32–58], blok `setContent` digunakan untuk menyusun UI aplikasi dengan Jetpack Compose; `isDarkModeEnabled` diamati secara reaktif dari `viewModel.isDarkModeEnabled` menggunakan `collectAsState`, dengan nilai awal diambil dari `settingsManager.isDarkModeEnabled()`; `MListComposeTheme` menerapkan tema gelap atau terang berdasarkan nilai tersebut; di dalamnya, `Surface` digunakan sebagai wadah utama dengan ukuran penuh layar (`fillMaxSize()`) dan warna latar dari `MaterialTheme`; `navController` diinisialisasi untuk mengelola navigasi antar layar; `NavHost` didefinisikan dengan `startDestination` "movieList" dan dua `composable` yang mewakili rute "movieList" dan "settings"; masing-masing rute menampilkan `MovieListScreen` dan `SettingsScreen` dengan `viewModel` dan `navController` yang diteruskan sebagai parameter, memungkinkan interaksi dan navigasi antar tampilan.

Notes :

Untuk Cache Strategy Room yang digunakan adalah “Cache First – Network Second”

Aplikasi pertama-tama mencoba mengambil data dari cache lokal (Room). Jika data ditemukan, data tersebut segera ditampilkan untuk responsivitas yang cepat. Setelah itu, aplikasi melakukan permintaan ke API jaringan untuk mendapatkan data terbaru. Jika permintaan jaringan berhasil, data lokal di cache akan diperbarui dengan data baru dari API, dan UI kemudian akan menampilkan data yang paling mutakhir dari cache yang sudah diperbarui. Jika permintaan jaringan gagal (misalnya karena tidak ada koneksi internet), tetapi ada data yang sudah tersimpan di cache, aplikasi akan tetap menampilkan data dari cache tersebut sambil memberikan notifikasi error bahwa data tidak dapat diperbarui.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Akmlsybn/Praktikum-Mobile>